Taller Bloque 7 · Editorial: UML, API REST en tres capas, MongoDB

Nov 11, 2026 · @Laura Meza

Universidad Cooperativa de Colombia · Campus Pasto · Diseño de Software

Integrantes: Laura Sofía Meza Reinoso · Juan Diego Moreno Revelo

Propósito y modelo de partida

El taller analiza cómo el modelo de clases de **Editorial** (`Publicacion`, `Libro`, `Disco`) se convierte en una API REST de tres capas (**Controller → Service → Repository**), cómo MongoDB guarda sus datos.

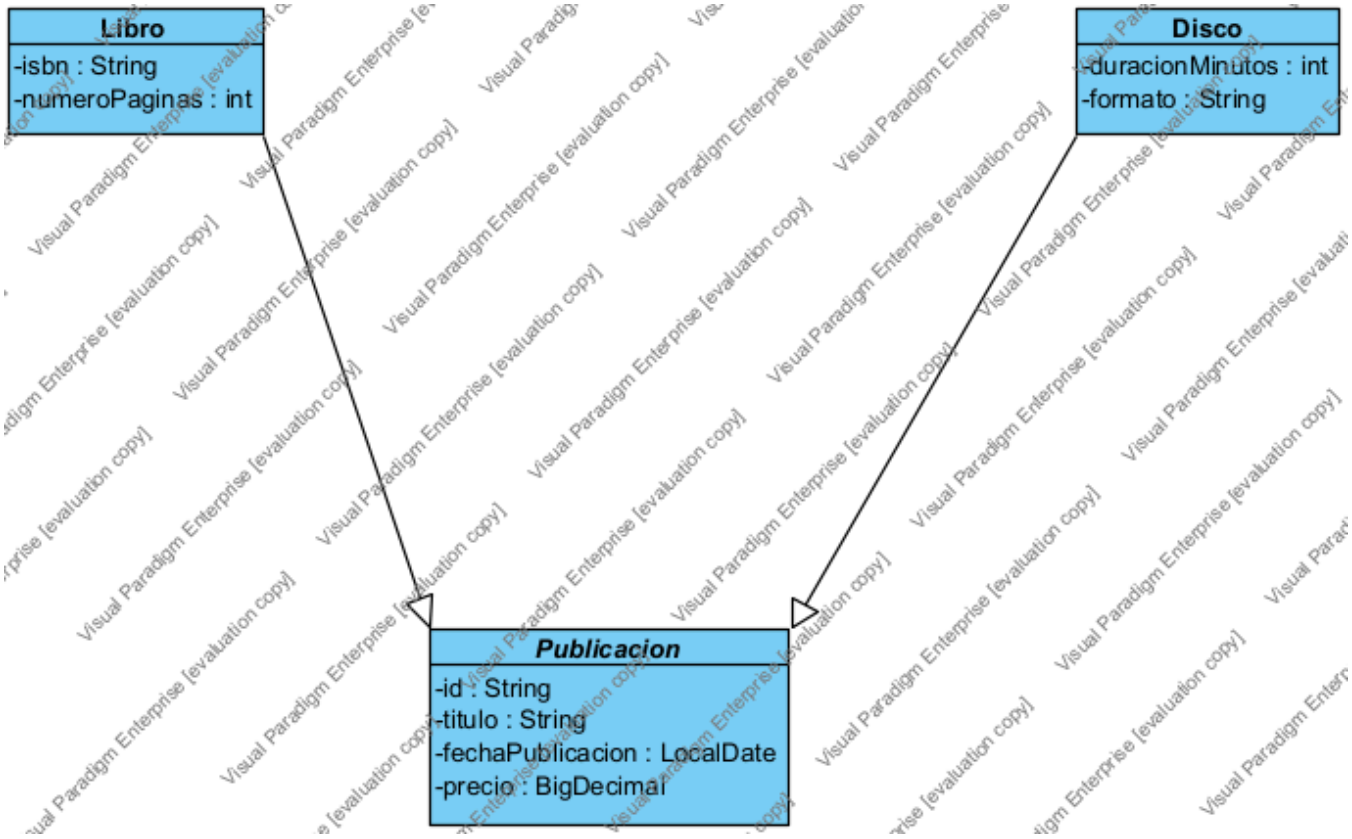


Diagrama 1 (Visual Paradigm): modelo de dominio Editorial.

- `Publicacion` es **abstracta** y concentra los datos comunes: `id`, `titulo`, `fechaPublicacion`, `precio`.
- `Libro` añade `isbn` y `numeroPaginas`; `Disco` añade `duracionMinutos` y `formato`.

- La flecha con **triángulo hueco** es una **generalización (herencia)**, no asociación ni composición.
- **¿Por qué no repetir `titulo` y `precio` en cada subclase?** Duplicaríamos código y reglas: un cambio en el precio o una validación del título habría que hacerlo en cada subclase y podrían quedar inconsistentes. Con la herencia el cambio se hace en un solo lugar y cualquier `Libro` o `Disco` se puede tratar como `Publicacion` (polimorfismo), por ejemplo al listar todo el catálogo.

1. Actividad previa sin IA (sección 8)

1.1 Tres cajas: Controller, Service y Repository

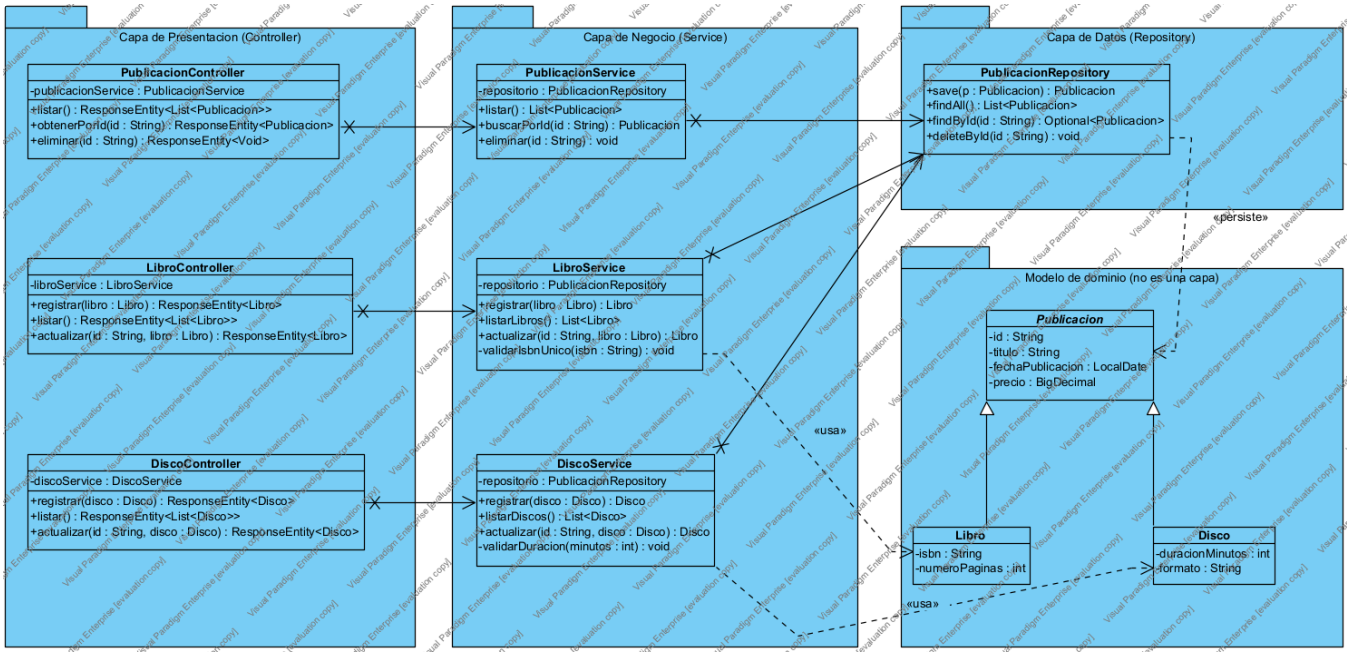


Diagrama 2 (Visual Paradigm): diseño por tres capas de la API Editorial.

Caja (capa)	Clases de Editorial	Responsabilidad
Controller (presentación)	`PublicacionController`, `LibroController`, `DiscoController`	Recibir la petición HTTP, validar el formato básico y devolver la respuesta con su código (200, 201, 204, 400, 404, 409).

Caja (capa)	Clases de Editorial	Responsabilidad
Service (negocio)	`PublicacionService`, `LibroService`, `DiscoService`	Coordinar el caso de uso y aplicar las reglas de negocio (ISBN único, precio positivo, duración válida).
Repository (datos)	`PublicacionRepository` (uno solo para toda la jerarquía)	Persistir y recuperar documentos de la colección `publicaciones` en MongoDB.
Modelo de dominio (no es una capa)	`Publicacion`, `Libro`, `Disco`	Datos que viajan entre las capas.

- **Asociación** (línea continua) Controller → Service y Service → Repository: cada clase conserva una referencia a su colaborador (por ejemplo `- libroService : LibroService`).
- **Dependencia** (línea punteada) hacia el modelo: las capas solo usan temporalmente `Libro`, `Disco` o `Publicacion`. No se confunde con la herencia del dominio.
- Ruta de un registro: **Cliente HTTP → LibroController → LibroService → PublicacionRepository → MongoDB**; la respuesta vuelve por el camino inverso.

1.2 ¿Quién valida una regla de negocio y quién guarda un documento?

- **La regla de negocio la valida el Service.** Al registrar un libro, `LibroService` verifica que el ISBN no exista ya (si existe, la API responde **409 Conflict**) y que el precio sea mayor que cero. El Controller solo revisa que la petición esté bien formada (JSON válido, campos obligatorios; si no, **400 Bad Request**).
- **El documento lo guarda el Repository.** `PublicacionRepository` (extiende `MongoRepository<Publicacion, String>`) ejecuta `save(...)` y MongoDB lo almacena en `publicaciones`. Ni el Controller ni el Service escriben consultas.

Así cada capa tiene una sola razón para cambiar: el protocolo cambia el Controller, una regla cambia el Service y la base de datos cambia el Repository.

1.3 ¿Qué cambia y qué permanece si añadimos Revista?

Qué cambia (se agrega):

1. **Modelo:** `Revista` hereda de `Publicacion` y solo declara lo propio: `issn`, `numeroEdicion`, `periodicidad`.
2. **Negocio:** `RevistaService` con sus reglas (ISSN válido y único, número de edición positivo).
3. **Presentación:** `RevistaController` con la ruta `/api/revistas` (POST, GET, PUT).
4. **Datos (solo contenido):** documentos nuevos con `"tipo": "REVISTA"` en la misma colección `publicaciones`.

Qué permanece igual:

- `Publicacion`, `Libro` y `Disco` no se modifican; tampoco sus Controller y Service.
- `PublicacionRepository` no cambia: trabaja con `Publicacion`, así que también guarda revistas.
- MongoDB, la base y la colección siguen iguales; no hay que migrar documentos.
- `PublicacionController` lista todo el catálogo, ahora con revistas, sin tocar su código (polimorfismo).

El diseño queda abierto a la extensión y cerrado a la modificación: añadir un subtipo es agregar clases, no reescribir las existentes.

2. Actividad opcional con IA (sección 9)

Petición a la IA: "Critica la distribución de responsabilidades del ejemplo Editorial (tres Controllers → tres Services → PublicacionRepository → MongoDB)".

Resumen de la crítica (parafraseada): (1) un solo repositorio con `findAll()` mezcla libros y discos, conviene consultar por tipo; (2) centralizar el manejo de errores con `@RestControllerAdvice`; (3) crear interfaz + clase `Impl` para cada Service y DTO + mapper por subtipo; (4) separar Libro y Disco en microservicios a futuro.

Recomendación	Decisión	Justificación del equipo
Consultas por tipo en el repositorio (`findByTipo`)	**Acertada**	Si `listarLibros()` usa `findAll()`, recibe también discos y debe filtrarlos a mano. La consulta por tipo deja el acceso a datos en el Repository y al Service solo con la regla.

Recomendación	Decisión	Justificación del equipo
Manejo centralizado de errores	**Acertada**	Hace explícito el contrato de la API: el Service lanza "ISBN duplicado" y un único punto lo traduce a **409 Conflict**. Evita repetir `try/catch` y que el Controller acumule lógica.
Interfaz + `Impl` por Service y DTO/mapper por subtipo	**Innecesaria en un curso introductorio**	Duplica las clases (de 3 a 6 services, más DTOs y mappers) sin una segunda implementación que lo justifique. Para entender las tres capas son más claras las clases concretas.
Microservicios por subtipo	Fuera de alcance	Controller, Service y Repository son capas lógicas en el mismo proceso Java; separar servidores no está en el escenario.

La crítica de la IA se usó solo como insumo; las decisiones y justificaciones son del equipo.

3. Autoevaluación (sección 10)

1. **¿En qué se diferencian Web e Internet?** Internet es la infraestructura de red (cables, enrutadores, TCP/IP) que conecta computadores. La Web es uno de los servicios que funciona sobre ella: recursos (páginas, APIs) a los que se accede con HTTP/HTTPS. El correo o la conexión a MongoDB también usan Internet y no son la Web.
2. **¿Qué diferencia existe entre HTTP y REST?** HTTP es un protocolo de mensajes (petición: método, ruta, encabezados, cuerpo; respuesta: código, encabezados, cuerpo). REST es un estilo arquitectónico: las rutas identifican recursos (`/api/libros/{id}`), los métodos expresan la operación y la respuesta representa el estado del recurso. REST usa HTTP, pero no es solo "usar JSON" o "hacer CRUD".
3. **¿Por qué GET y POST no tienen el mismo propósito?** GET consulta sin modificar (seguro e idempotente): `GET /api/publicaciones` → **200 OK**. POST crea un recurso con datos en el cuerpo; repetirlo crea otro: `POST /api/libros` → **201 Created** con el id. Usar GET para crear rompería el contrato y podría duplicar datos al recargar.

4. **¿Qué guarda una colección de MongoDB?** Documentos (estructuras tipo JSON/BSON con campos y valores), cada uno con su `_id`. En Editorial, `publicaciones` guarda libros y discos; como pueden tener campos distintos, el campo `tipo` indica el subtipo.
5. **¿Qué relación UML conecta Libro con Publicacion?** Una **generalización (herencia)**: línea continua con triángulo hueco hacia `Publicacion`. Se lee "un Libro es una Publicación".
6. **¿Por qué el Controller no debería implementar toda la lógica de negocio?** Su responsabilidad es la comunicación HTTP. Con reglas y consultas adentro, la lógica queda atada al protocolo, se duplica entre controladores, es difícil de probar sin servidor y cualquier cambio de regla toca la presentación. Separarla en Service y Repository da alta cohesión y bajo acoplamiento.

4. Contrato mínimo de la API y documentos MongoDB

Antes de programar, cada operación queda definida con método, ruta, entrada, éxito y errores previstos.

Método y ruta	Parámetros	Entrada (JSON)	Éxito	Errores previstos
`GET /api/publicaciones`	—	—	200 OK + lista	500
`GET /api/publicaciones/{id}`	`id` (ruta)	—	200 OK + publicación	404 si no existe
`DELETE /api/publicaciones/{id}`	`id` (ruta)	—	204 No Content	404 si no existe
`POST /api/libros`	—	titulo, fechaPublicacion, precio, isbn, numeroPaginas	201 Created + libro con id	400 datos inválidos · 409 ISBN repetido
`GET /api/libros`	—	—	200 OK + lista de libros	500
`PUT /api/libros/{id}`	`id` (ruta)	libro completo	200 OK + libro	400 · 404

Método y ruta	Parámetros	Entrada (JSON)	Éxito	Errores previstos
`POST /api/discos`	—	titulo, fechaPublicacion, precio, duracionMinutos, formato	201 Created + disco con id	400 datos inválidos
`GET /api/discos`	—	—	200 OK + lista de discos	500
`PUT /api/discos/{id}`	`id` (ruta)	disco completo	200 OK + disco	400 · 404

Documentos de la colección `publicaciones` (una sola colección con campo `tipo`):

```
{ "_id": "p01", "tipo": "LIBRO", "titulo": "Diseño de Software",
"fechaPublicacion": "2026-03-10", "precio": 65000, "isbn": "978-958-00-0000-
1", "numeroPaginas": 280 }
{ "_id": "p02", "tipo": "DISCO", "titulo": "Sonidos del Galeras",
"fechaPublicacion": "2025-11-20", "precio": 42000, "duracionMinutos": 48,
"formato": "CD" }
{ "_id": "p03", "tipo": "REVISTA", "titulo": "Ingeniería Hoy",
"fechaPublicacion": "2026-10-01", "precio": 18000, "issn": "2346-0000",
"numeroEdicion": 12, "periodicidad": "Mensual" }
```

Flujo "registrar un libro": `POST /api/libros` → `LibroController` interpreta y delega → `LibroService` valida reglas → `PublicacionRepository.save()` persiste en MongoDB → la API responde **201 Created** con el id.

Referencias

- Material docente: *Lectura previa Bloque 7*, Diseño de Software.
- MDN Web Docs – HTTP
- Spring Data MongoDB – Reference
- MongoDB – Documentación